DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institution Affiliated to VTU, Belagavi)

SHAVIGE MALLESHWARA HILLS, K.S.LAYOUT, BANGALORE-560078

Department of Computer Science and Engineering (Cyber Security)

2023-2024

IV Semester

Workspace Security Practices

Laboratory Manual

Compiled by,

Dr. Deepthi V S

WORKSPACE SECURITY PRACTICES

Course Code:22CYL471	**Credits: 01**	**L: P: T: S: 0: 2: 0: 0**	**Total Hours: 12**
CIE Marks: 50	**SEE Marks: 50**		**Exam Hours: 03**

Course objectives:

1. Understand various techniques for providing workspace security to secure file & password
2. Apply appropriate tools for network packet sniffing, attack detection and security incident reporting

Sl No.	EXPERIMENTS
	PART A
1	Explore Compare It Tool to Compare of two files for Forensic Investigation.
2	Explore the Snow Tool for hiding the information in Text File.
3	Write a program to illustrate Buffer overflow attack.
4	Write the steps to Download a website using Website Copier tool (HTTrack) to perform passive reconnaissance
5	Analyze and scan the System Vulnerabilities using MicrosoftBaseline Security Analyzer (MBSA) Tool.
	PART B
1	**Password Strength Checker:** Develop a Python script that assesses the strength of passwords based on criteria such as length, complexity, and the presence of common patterns.
2	**File Encryption and Decryption:** Create a Python program that encrypts and decrypts files using cryptographic algorithms such as AES (Advanced Encryption Standard).
3	**Network Packet Sniffer:** Build a packet sniffing tool in Python that captures and analyses network traffic on a local network.
4	**Brute Force Attack Detector:** Develop a Python script that detects and logs brute force login attempts on a server or application.
5	**Security Incident Reporting Tool:** Develop a Python-based incident reporting tool where employees can report security incidents such as lost devices, suspicious emails, or physical security breaches.

Course Outcomes: After completion of the course, the graduates will be able to:

1. Implement python program to assess the strength of password for complexity measures
2. Apply cryptographic algorithms for a given file
3. Apply appropriate python modules for network packet sniffing, attack detection and security incident reporting.

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

Semester-End Examination:

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.
- All laboratory experiments are to be included for practical examination.
- Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.
- Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.
- General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure and result in -60%, Viva-voce 20% of maximum marks.
- Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero.
- The minimum duration of SEE is 02 hours

Continuous Internal Evaluation: for the practical component

On completion of every experiment/program in the laboratory, the students shall be evaluated and marks shall, be awarded on the same day. The 30 marks are for conducting the experiment and preparation of the laboratory record is for 10 marks, the other 10 marks shall be for the test conducted at the end of the semester.

Typical Evaluation pattern for PCC course is shown in Table 1.

Component		Max Marks	Total Marks	Min Marks
CIE Lab	Practical(Conduction)	30	50	12
	Practical(Record)	10		-
	Practical(Test)	10		8
Total CIE Practical			**50**	**20**
SEE	Semester End Examination	50	50	18
CIE+SEE			**100**	**40**

PART – A

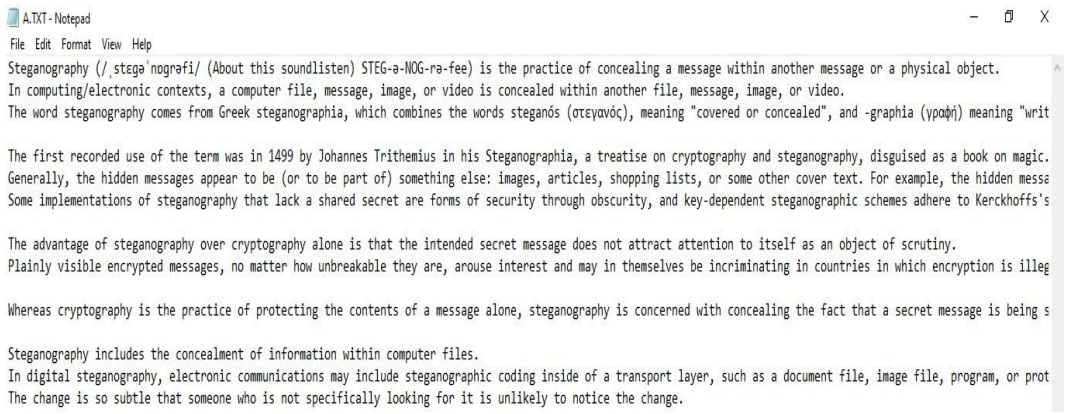
EXPT.No. 1	Explore Compare It Tool to Compare of two files for ForensicInvestigation.	DATE:

AIM:
The main aim is to comparison of two files for forensics investigation by COMPARE IT tool

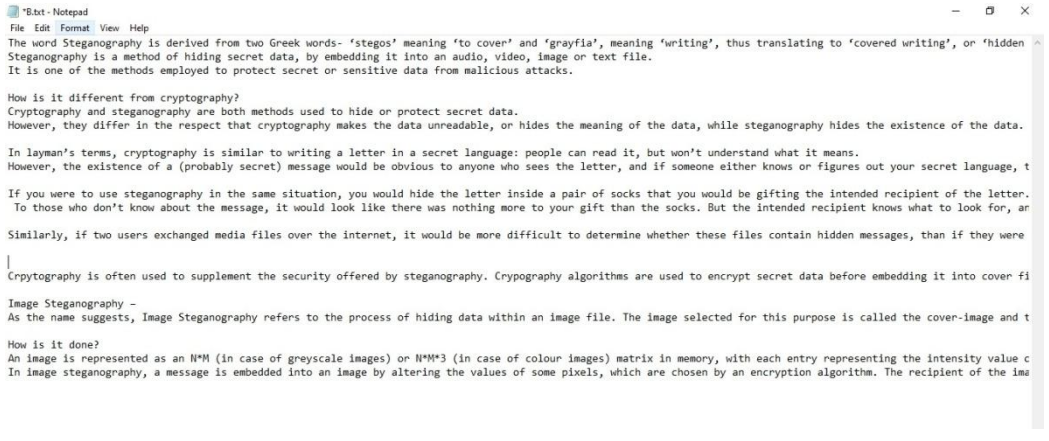
PROCEDURE:

- COMPARE IT is software that displays 2 files side by side, with colored differences sections to simplify analyzing. You can move changes between files with a single mouse click or keystroke, and of course, you have the ability to edit files directly in comparison window.
- It can make colored printout of differences report, exactly as it's on the screen. First of all, install the Compare It from the Link given below. http://www.grigsoft.com/wincmp3.htm it is a 1.7 Mb Software package Click on Compare It Tool, It will show a window to select the files to be compared.
- First, select the first file and click on open and then select the second file and click on open.

STEP 1: open the notepad and create a first text file with the extension .txt and save with a file name

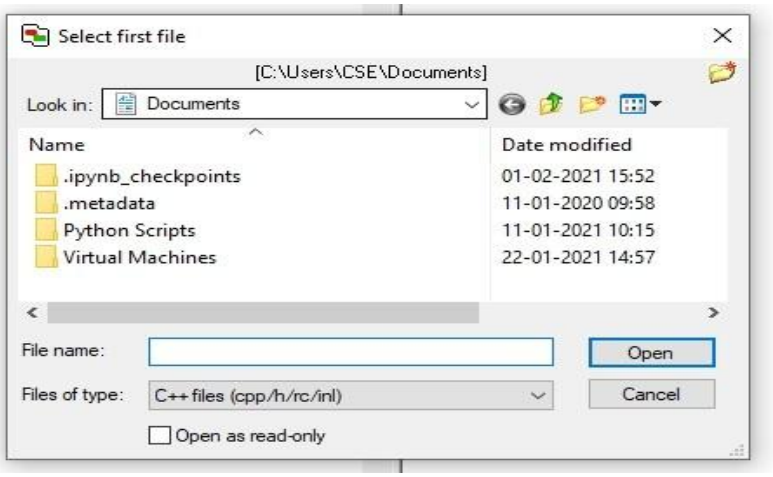


Step 2: create a second text file with the extension .txt

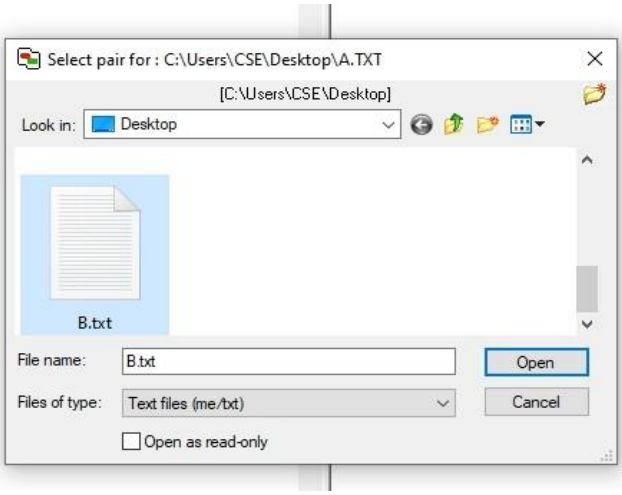


Step 3: Download the compare it tool install the Compare It from the Link given below. http://www.grigsoft.com/wincmp3.htm it is a 1.7 Mb Software package Click on Compare It Tool, It will show a window to select the files to be compared.

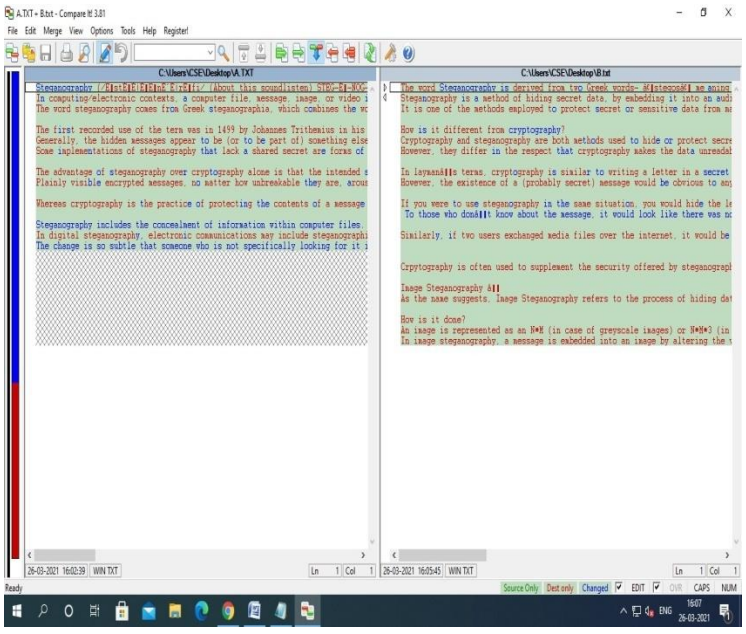
Step 4: Upload the first file to the compare it tool



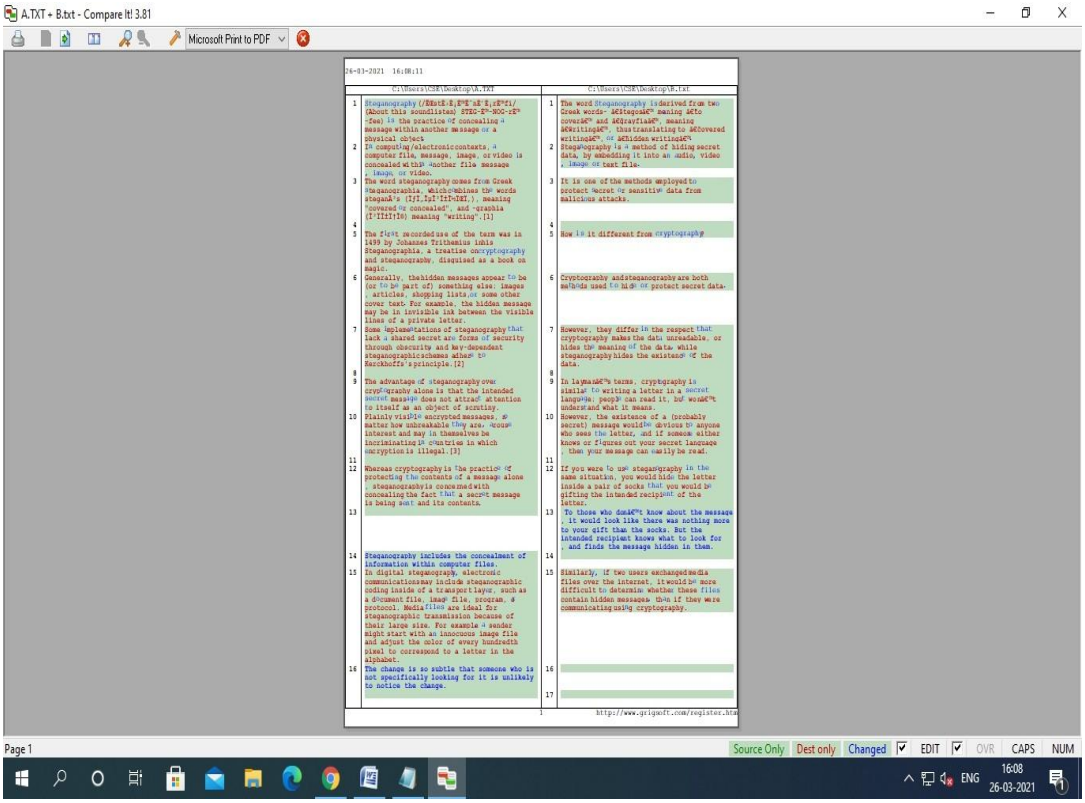
Step 5: upload the second file to the compare it tool



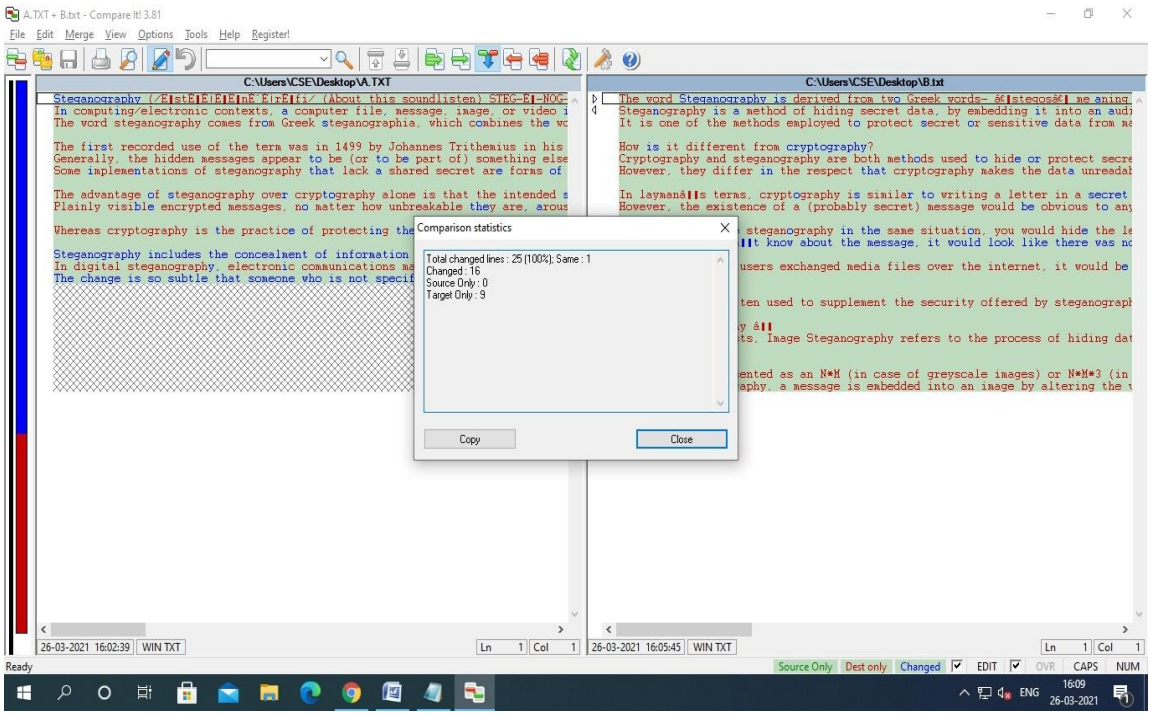
Step 6: Displays 2 files side by side, with colored differences sections to simplify analyzing. You can move changes between files with a single mouse click or keystroke



STEP 7: It also gives you Print report of the difference in the file as follows



STEP 8: the comparison result is get display.



RESULT:

The main aim is to comparison of two files for forensics investigation byCOMPARE IT tool is executed successfully.

EXPT.No. 2	Explore the Snow Tool for hiding the information inText File	DATE:

AIM:

The main aim is to hide the information in the Text File Using SNOW TOOL- TextStenography

PROCEDURE:

1) Create a text File with some data in the same directory where SNOW Tool is installed.
2) In our Experiment Snow tool is installed in Desktop.

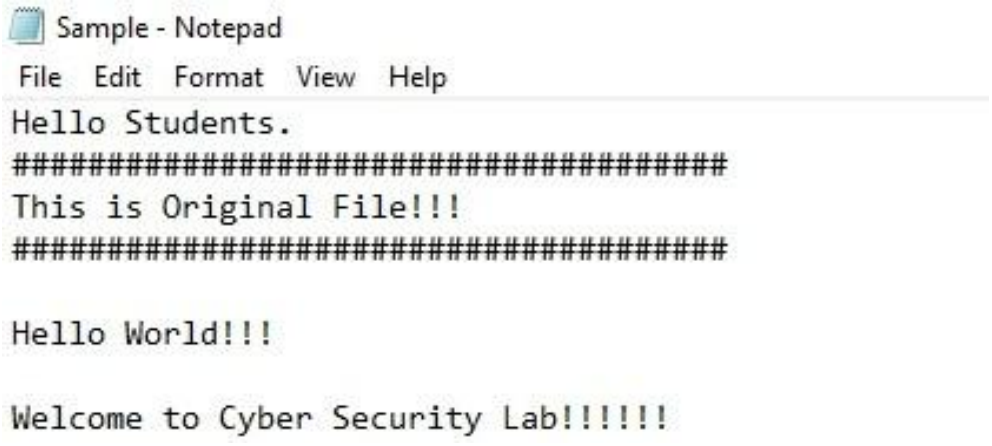


Figure: Text File

3) Go to the Command Prompt, Change the directory to run snow Tool

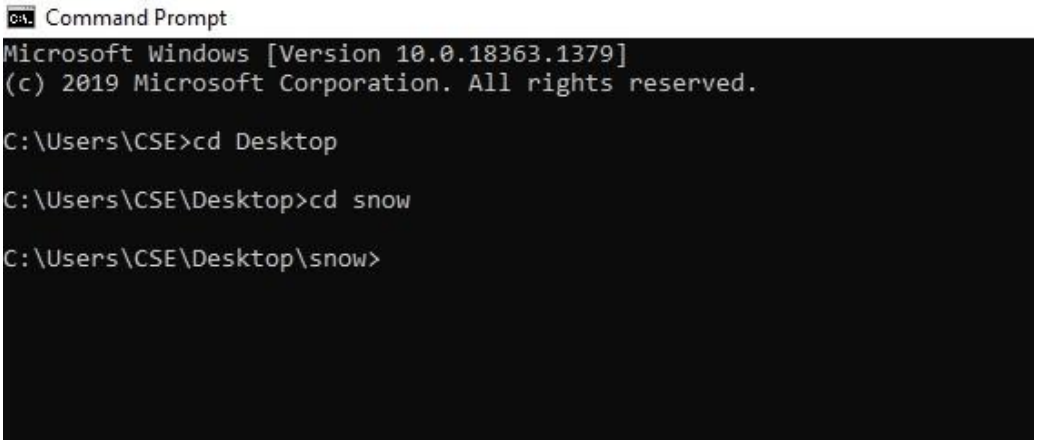


Figure: Changing the Directory

4) Type the Command:

snow -C -m “text to be hidden ” –p “password” <Source File><Destination File>

5) Example:

Snow -C -m “My Account number 1234567” –p “password123” Sample.txtTest.txt

The Source file is a Sample.txt file as shown above.
Destination file will becreated automatically and exact copy of source file containing hidden information.

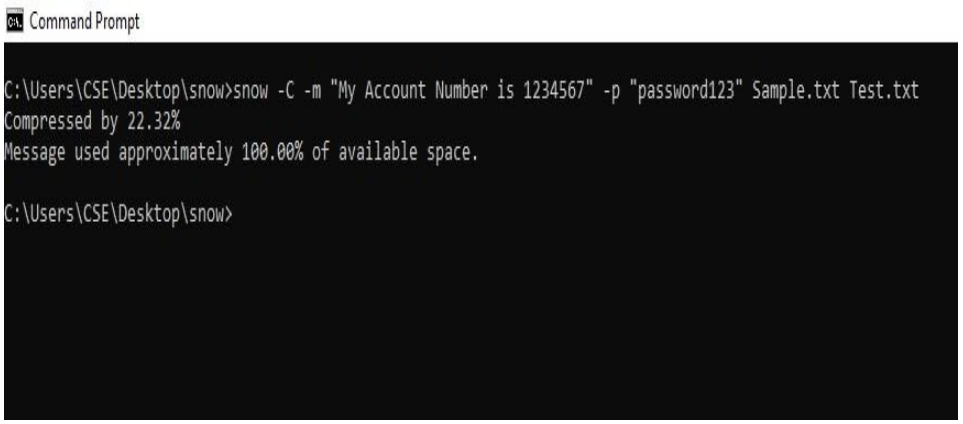


Figure: White Space Steganography using Snow Tool

6) **Go to the Directory:** You will find a new File by name Test.txt. Open the file

Figure: File Containing Hidden Encrypted Information

7) New file has the same text as an Original file (Sample.txt) without any hiddeninformation. This file can be sent to the target.
8) Recovering the Hidden Information :

On the Destination, the receiver can reveal information by using the commandsnow –C –p “password” <Destination File>

snow –C –p “password123” test.txt

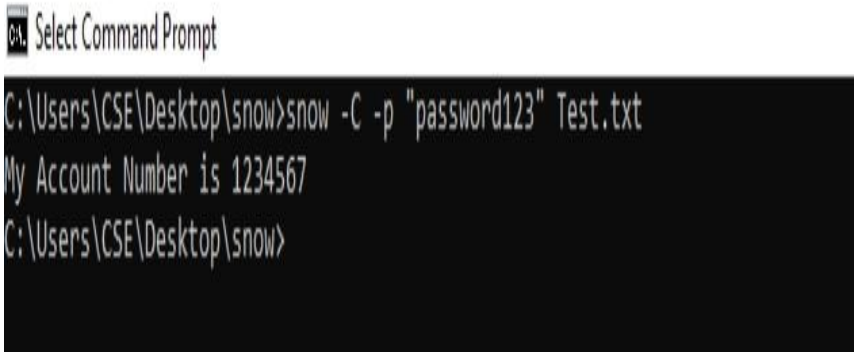


Figure: Decrypting File

As shown in the above figure, file decrypted, showing hidden information encrypted in the previous section

RESULT:

The main aim is to hide the information in the Text File Using SNOW TOOL - Text Stegnography is completed successfully.

EXPT.No. 3	Write a program to illustrate Buffer overflow attack	DATE:

AIM:

The main aim is to write a program to illustrate buffer overflow attack.

PROCEDURE:

A buffer overflow (or buffer overrun) occurs when the volume of data exceeds the storage capacity of the memory buffer. If the transaction overwrites executable code, it can cause the program to behave unpredictably and generate incorrect results, memory accesserrors, or crashes.

```
#include <stdio.h>
 #include <string.h>
int main(void)
{
char buff[15];
int pass = 0;
printf("\n Enter the password : \n");
gets(buff);
if(strcmp(buff, "thegeekstuff"))
{
        printf ("\n Wrong Password \n");
}
else
{
        printf ("\n Correct Password \n");
        pass = 1;
}
if(pass)
{
        /* Now Give root or admin rights to user*/
```

```
        printf ("\n Root privileges given to the user \n");
}
return 0;
}
```

The program above simulates scenario where a program expects a password from user and if the password is correct then it grants root privileges to the user.

Let's the run the program with correct password ie 'thegeekstuff' :

OUTPUT

RUN1

Enter the password :

thegeekstuff

Correct Password

Root privileges given to the user

This works as expected. The passwords match and root privileges are given. But do you know that there is a possibility of buffer overflow in this program. The gets() function does not check the array bounds and can even write string of length greater than the size of the buffer to which the string is written. Now, can you even imagine what can an attacker do with this kind of a loophole?

Here is an example :

RUN 2

Enter the password :

hhhhhhhhhhhhhhhhhhhh

Wrong Password

Root privileges given to the user

RESULT:

The main aim is to write a program to illustrate buffer overflow attack is completedsuccessfully

EXPT.No. 4	Write the steps to Download a website using Website Copier tool (HTTrack) to perform passive reconnaissance	DATE:

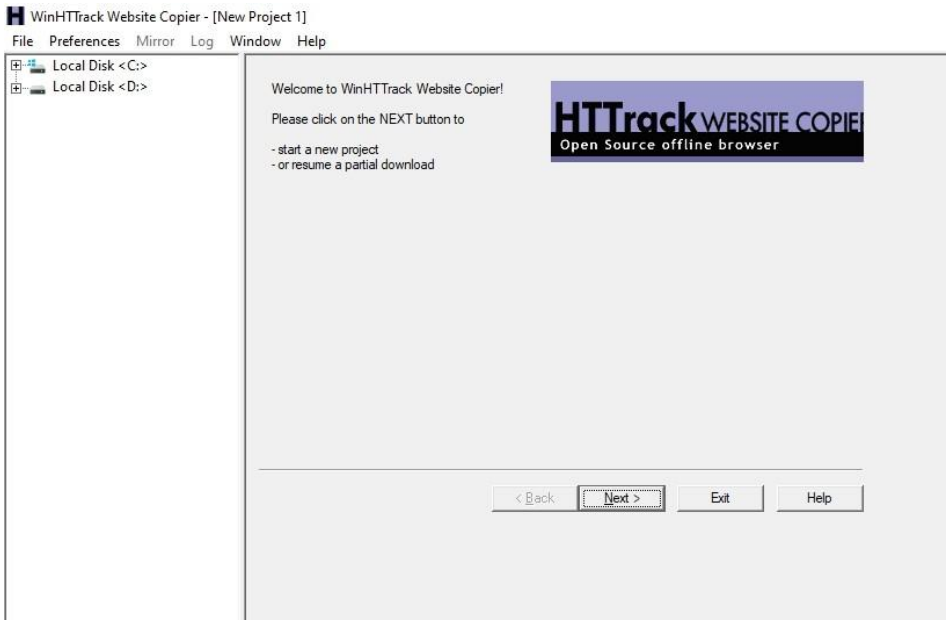
AIM:

The main aim is to downloading a website using website copier tool (HTTtack)

PROCEDURE:

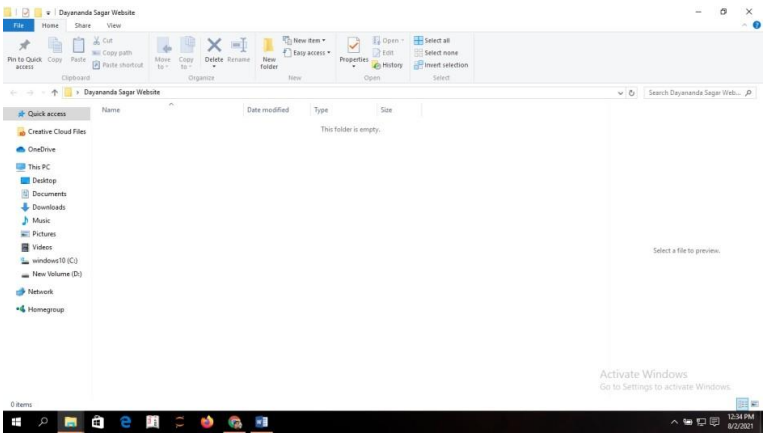
- HTTrack is a free (GPL, libre/free software) and easy-to-use offline browser utility. It allows you to download a World Wide Web site from the Internet to a local directory, building recursively all directories, getting HTML, images, and other files from the server to your computer.
- HTTrack arranges the original site's relative link-structure.
- Simply open a page of the "mirrored" website in your browser, and you can browse the site from link to link, as if you were viewing it online.
- HTTrack can also update an existing mirrored site, and resume interrupted downloads.HTTrack is fully configurable, and has an integrated help system.
- WinHTTrack is the Windows (from Windows 2000 to Windows 10 and above) release of HTTrack, and WebHTTrack the Linux/Unix/BSD release.

STEP 1: Install WinHTTrack

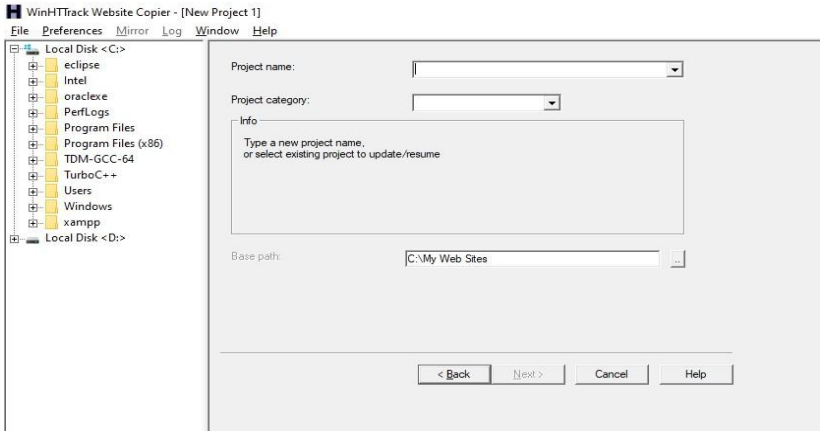


STEP 2: Create a folder on the Desktop and rename the folderFor Example: Folder name is “Dayananda Sagar Website”.

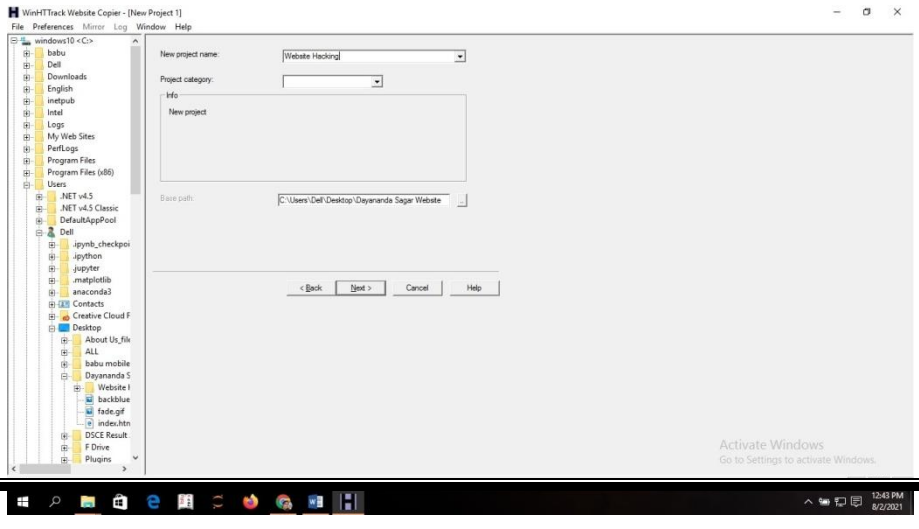
Open the folder “Dayananda Sagar Website”. The content of the folder is empty.



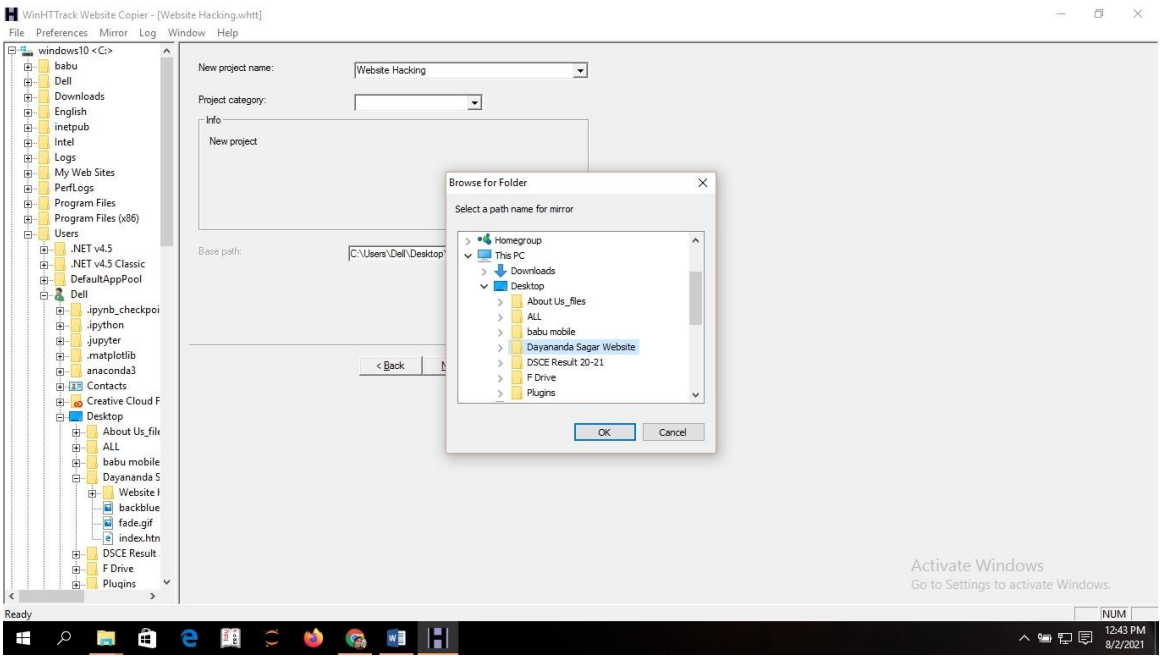
STEP 3: Select the new project from the file menu



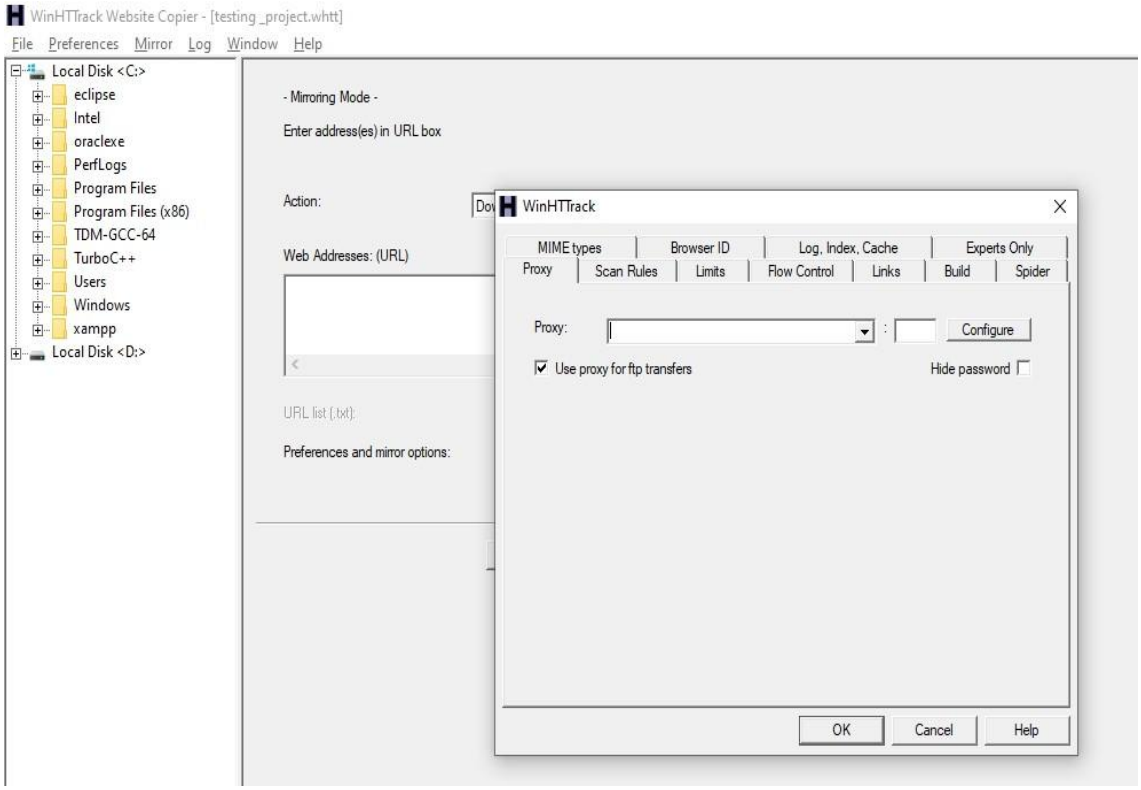
STEP 4: Enter the project name in new project field: **Example: Website hacking**



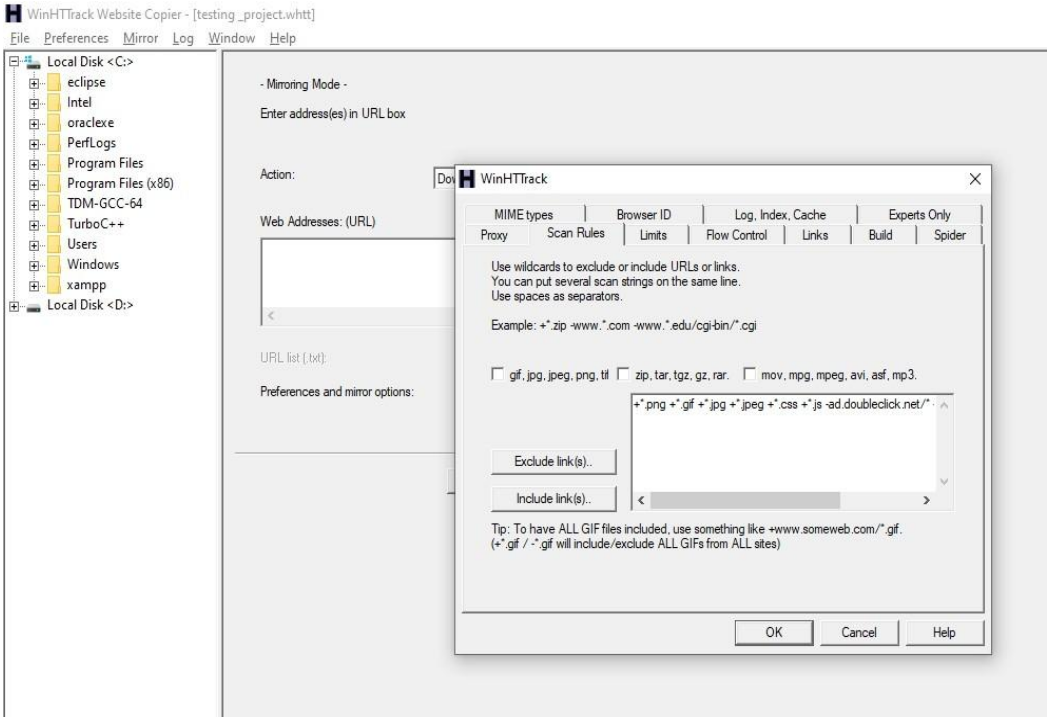
Step 5: Give the path where you need to download the files. In order to do this Clickon Desktop and then click the folder “Dayananda Sagar Website”. Press OK



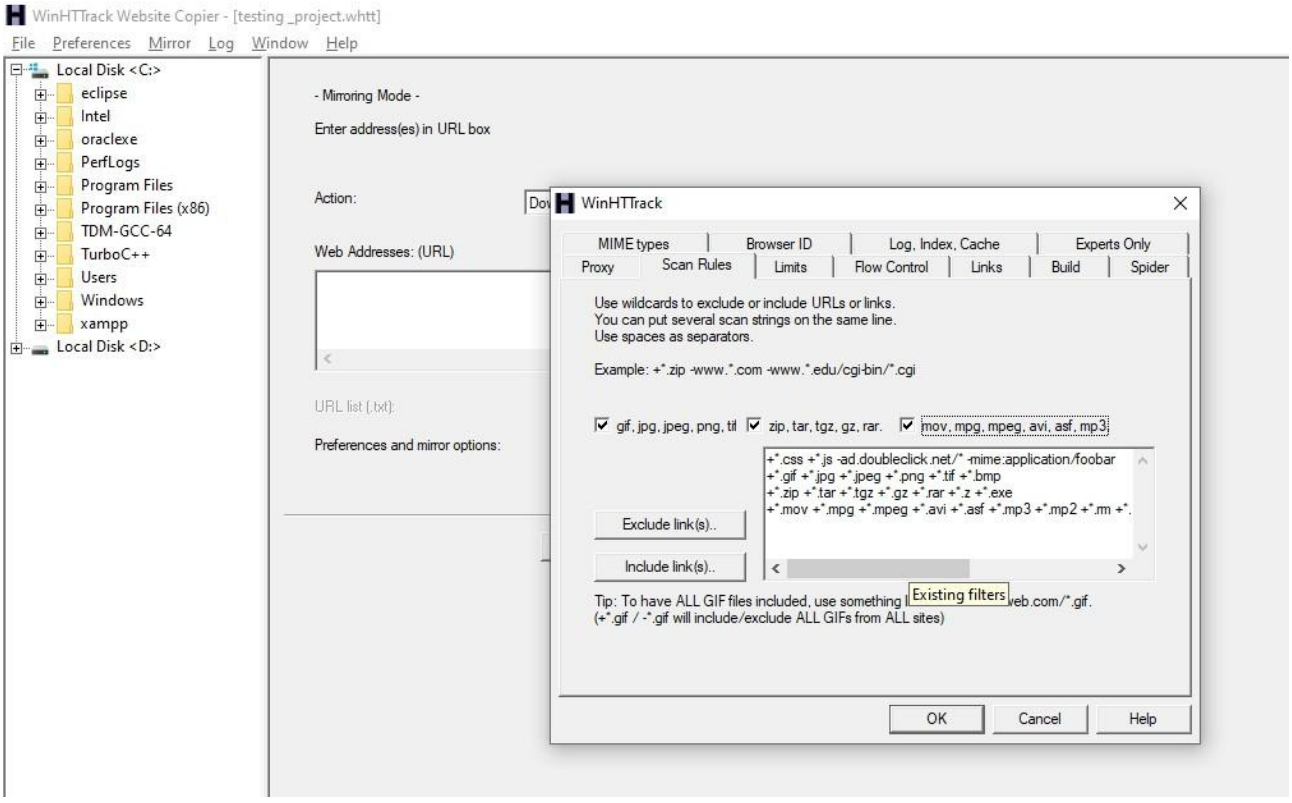
Step 6: WinHTTrack option window is opened select the scan rules



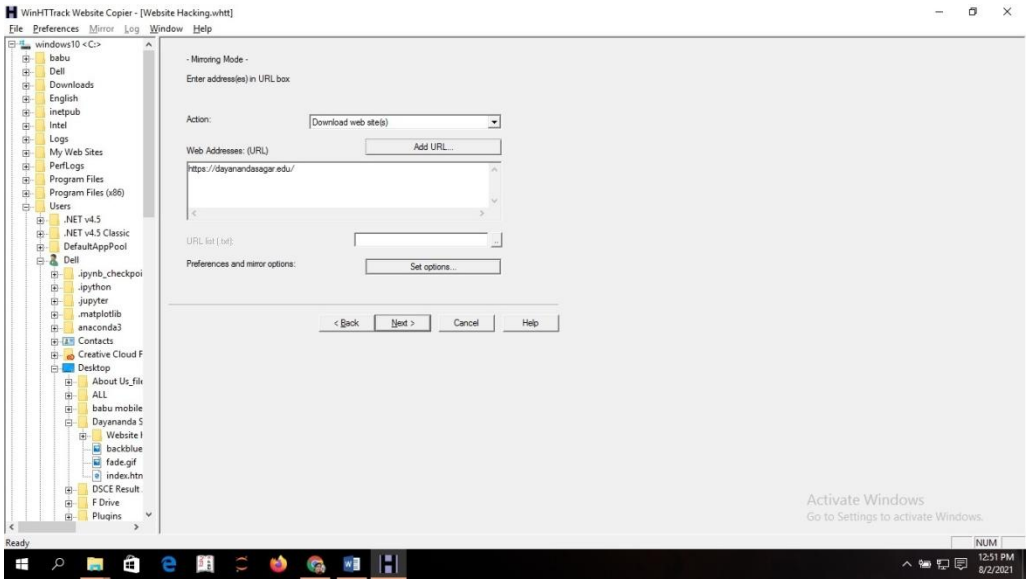
Step 7: Select all type of file to start the scan.



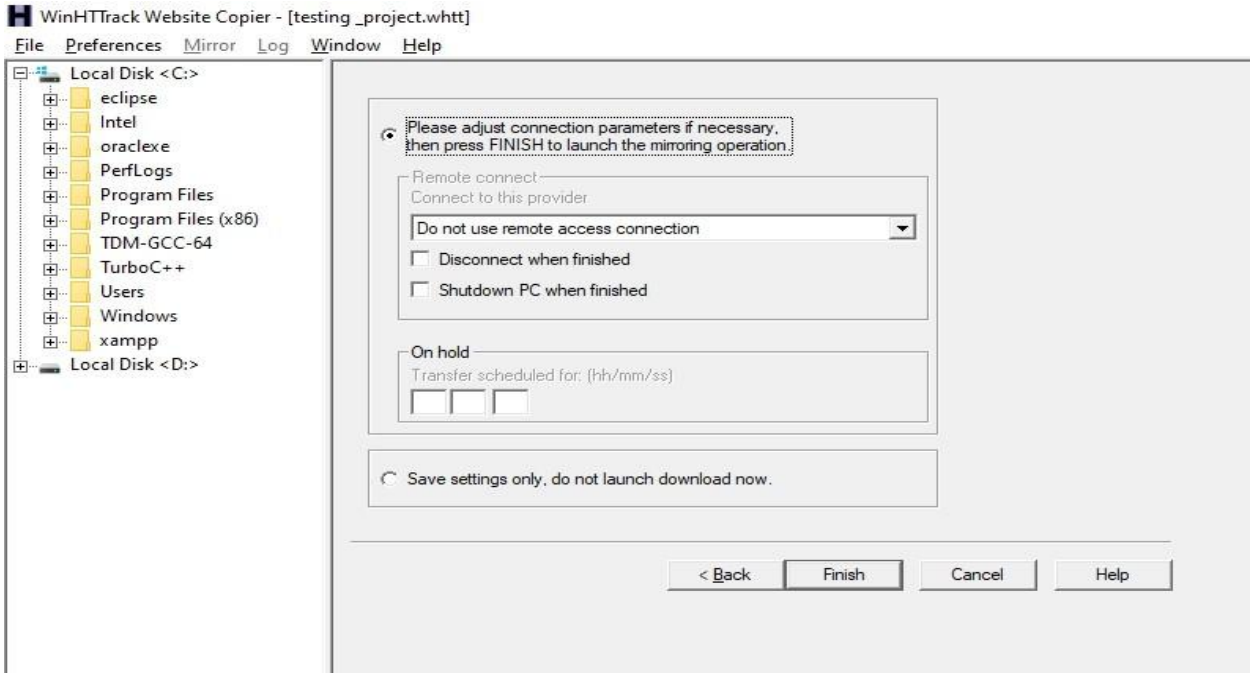
Step 8: Now all the extension is added for the scan.



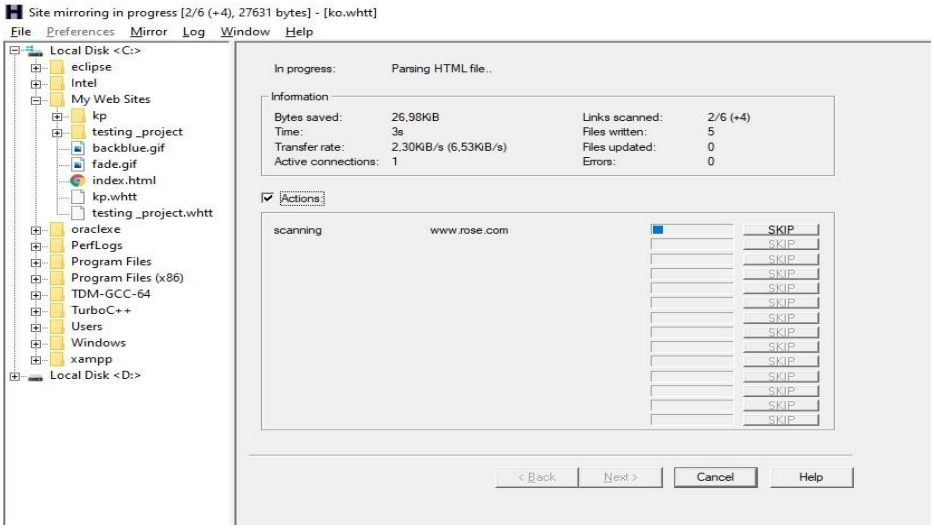
Step 9: Now type the URL address to scan



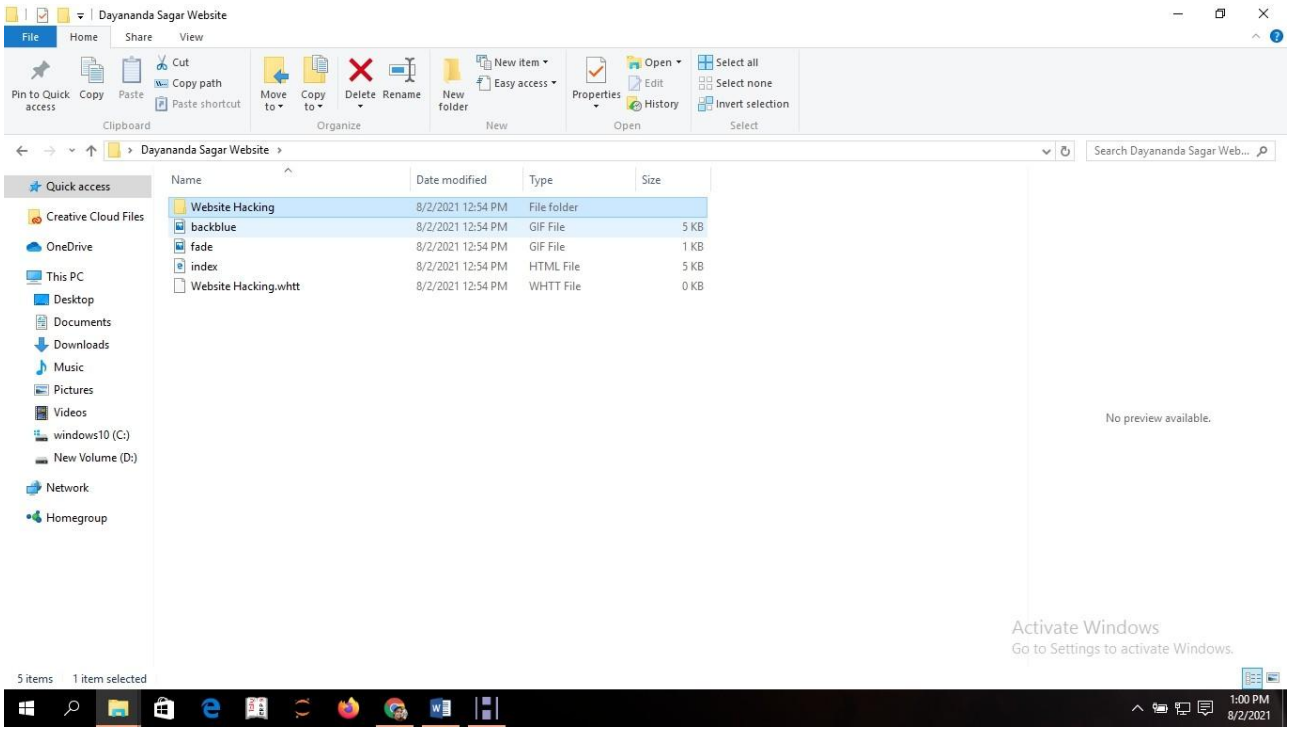
Step 10: Enable the connection adjustment if needed and click the finish button.



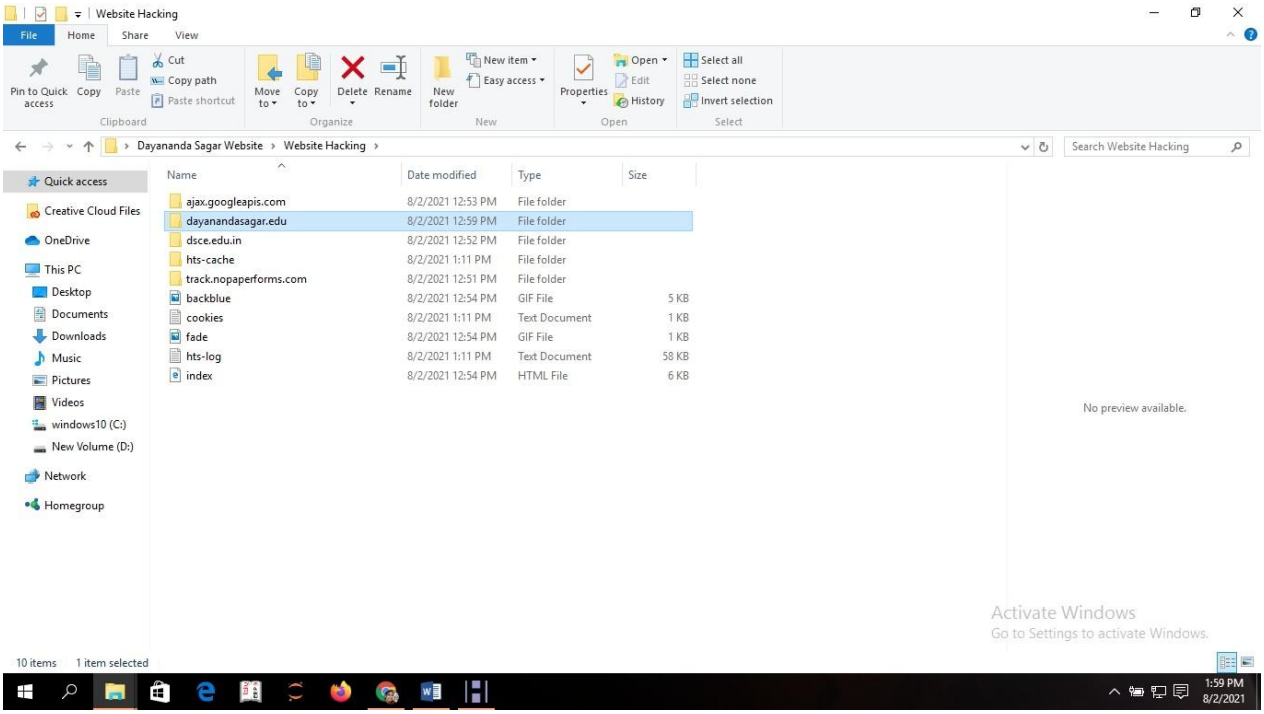
Step 11: mirroring process is get started



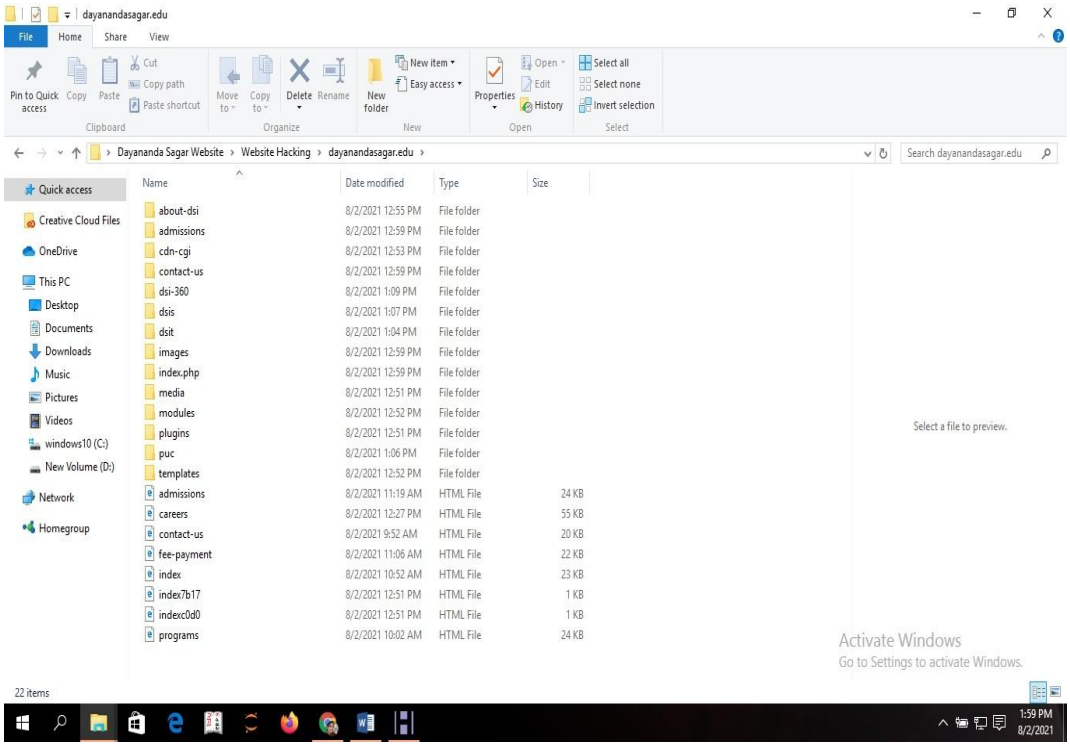
Step 12: The detail information about the URL will be fetched and saved in the folder "Dayananda Sagar Website". You can now open the folder where you can see the project name given as Website hacking as shown in Step 3.



Step 13: Click on Website hacking file, then the URL address dayanandasagar.edu file given in the Step 8 will be visible.

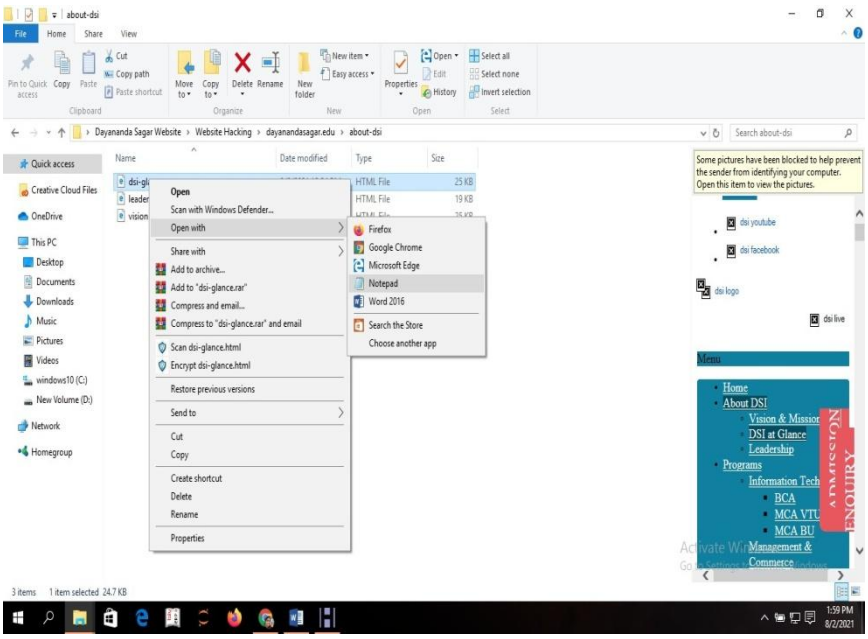


Step 14: Click on the file, dayanandasagar.edu. Now you can find all the files of theoriginal page of the Website.

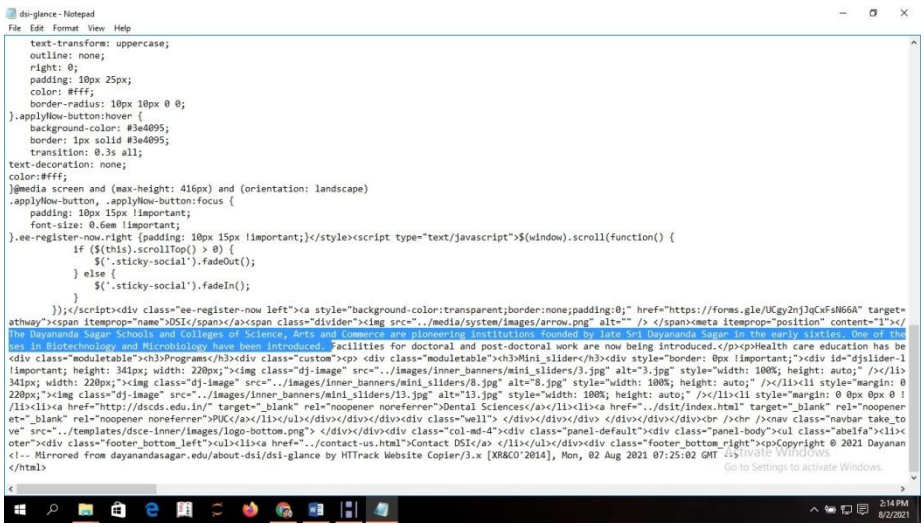


Step 15: Click on any file to alter the content: open with notepad.

Example: Click the file about-dsi. 3 Files are displayed. Any file can be opened in a notepad then changes can be done in the file.



Step 16: Contents of the pages is displayed. Now you can alter the Contents.



RESULT:

The main aim is to downloading a website using website copier tool (HTTtack) iscompleted successfully

EXPT.No. 5	Analyze and scan the System Vulnerabilities using Microsoft Baseline Security Analyzer (MBSA) Tool.	DATE:

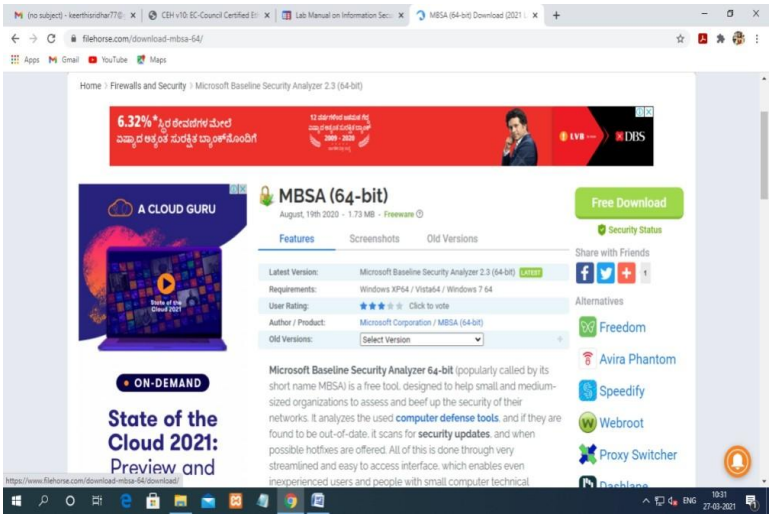
AIM:

The main aim is to scan the system vulnerabilities using Microsoft baseline security analyzer **(MBSA)**

PROCEDURE:

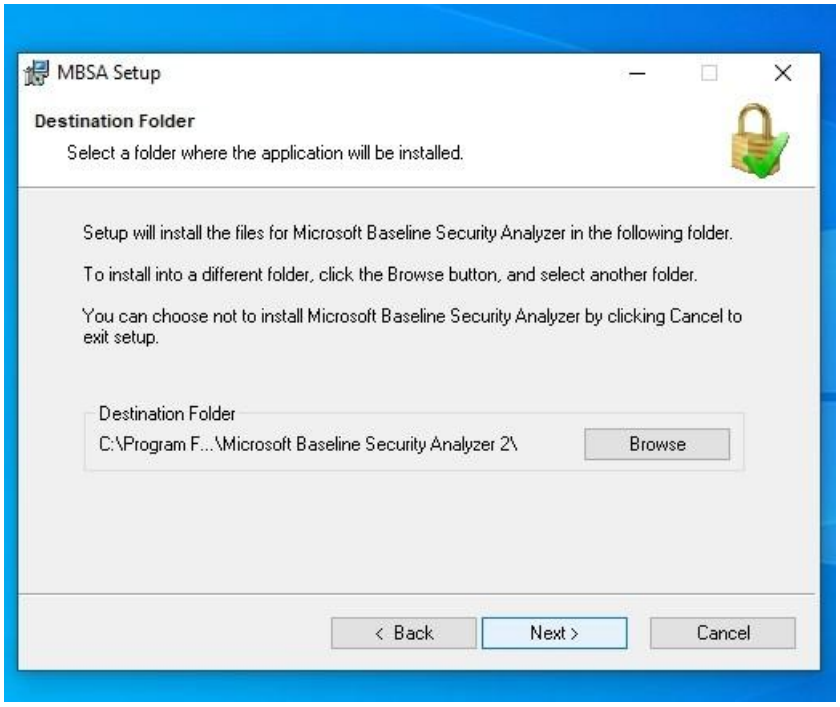
- Microsoft Baseline Security Analyzer (MBSA) is used to verify patch compliance. MBSA also performed several other security checks for Windows, IIS, and SQL Server.
- Unfortunately, the logic behind these additional checks had not been actively maintained since Windows XP and Windows Server 2003.
- Changes in the products since then rendered many of these security checks obsolete and some of their recommendations counterproductive.
- MBSA was largely used in situations where neither Microsoft Update nor a local WSUS or Configuration Manager server was available, or as a compliance tool to ensure that all security updates were deployed to a managed environment.
- While MBSA version 2.3 introduced supports for Windows Server 2012 R2 and Windows 8.1, it has since been deprecated and no longer developed. MBSA 2.3 is not updated to fully support Windows 10 and Windows Server 2016.

Step 1: download the Microsoft Baseline Security Analyzer (MBSA)

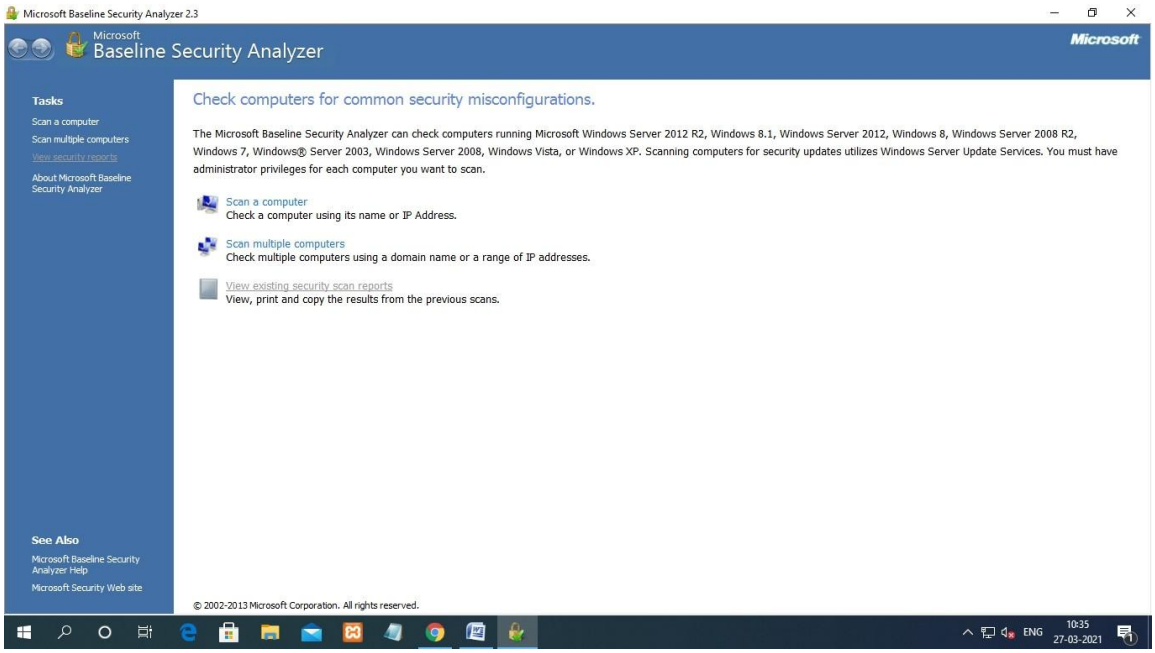


Step2: start and install the Microsoft Baseline Security Analyzer (MBSA) click next to start install

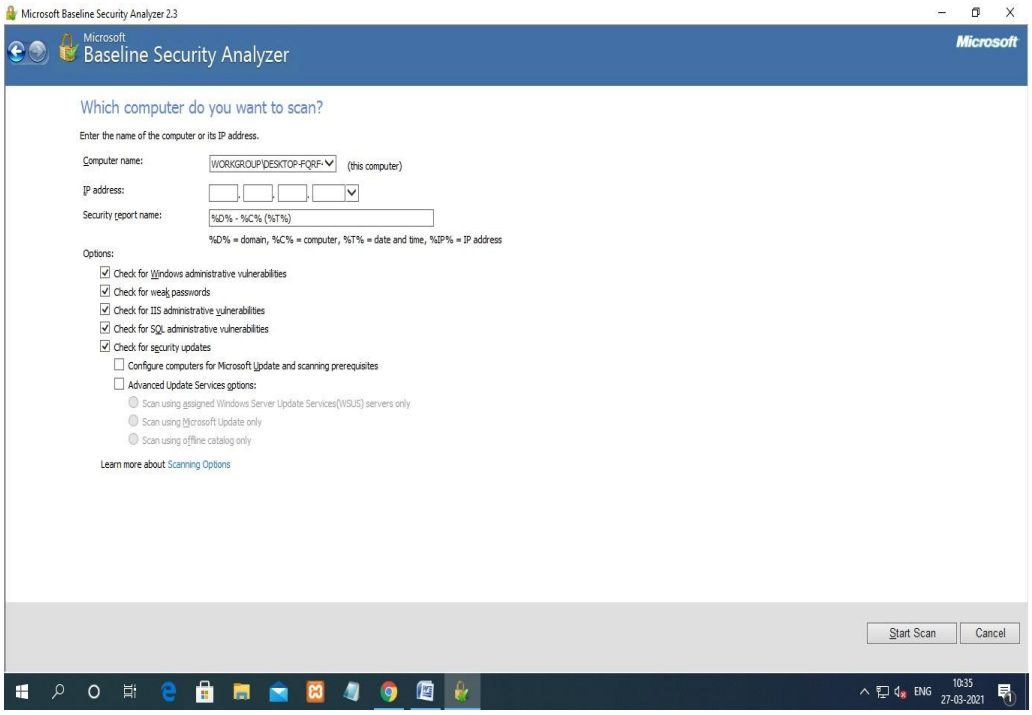
Step3: choose the location to install MBSA



STEP 4:click the scan a computer to start the MBSA

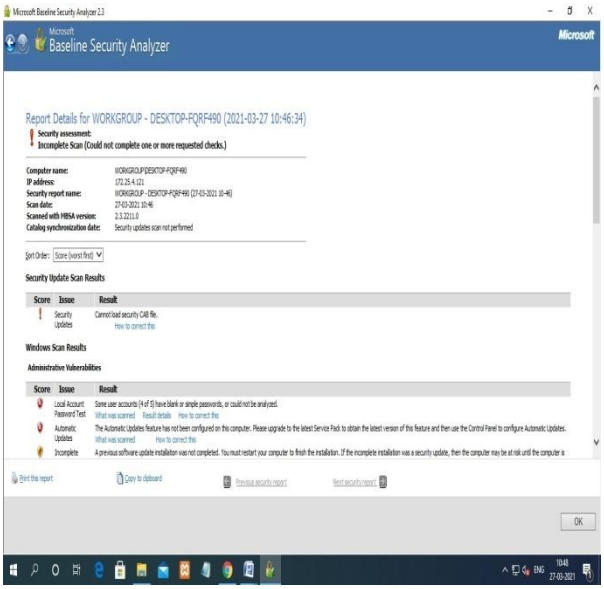


Step 5: provide the IP address and click start scan

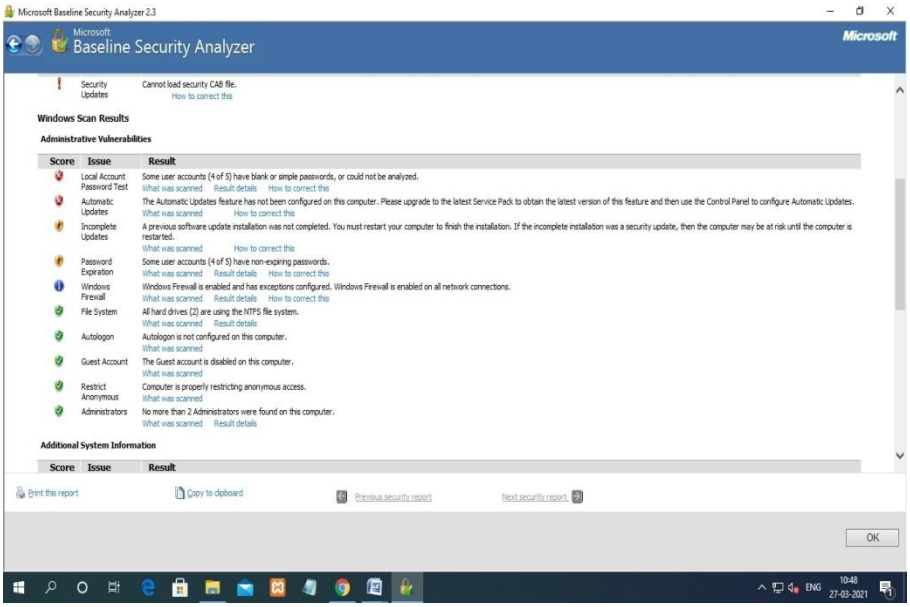


Step 7: The scanning process is GET STARTED

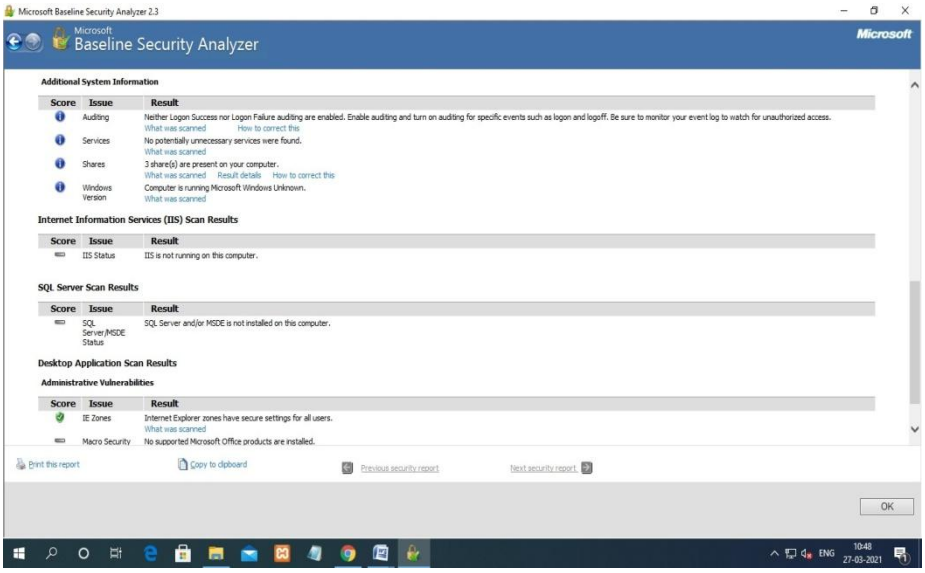
Step 8: The detail report is generated for the system



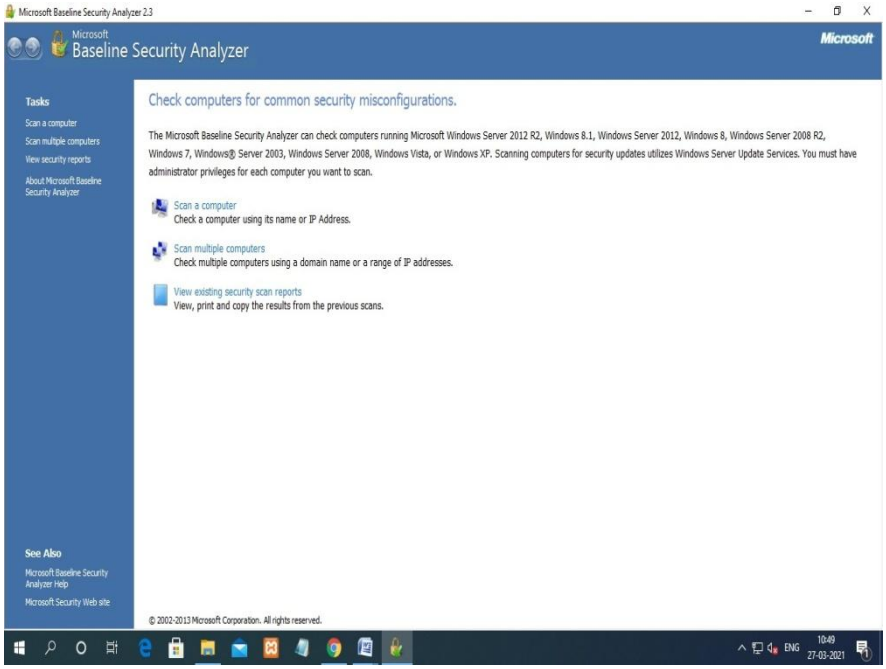
Result about the administrative vulnerabilities



Result about the additional system information, IIS scans result, desktop application



And also we can view the existing scan report



RESULT:

The main aim is to scan the system vulnerabilities using Microsoft baselinesecurity analyzer **(MBSA)** is completed successfully.

PART – B

EXPT.No. 6	**Password Strength Checker:** Develop a Python script that assesses the strength of passwords based on criteria such as length, complexity, and the presence of common patterns.	DATE:

```
import re
def password_strength(password):
    # Length check
    if len(password) < 8:
        return "Weak: Password is too short"

    # Complexity check
    has_lowercase = any(char.islower() for char in password)
    has_uppercase = any(char.isupper() for char in password)
    has_digit = any(char.isdigit() for char in password)
    has_special = any(char in '!@#$%^&*()-_=+[]{}|;:,.<>?`~' for char in password)

    complexity = sum([has_lowercase, has_uppercase, has_digit, has_special])

    if complexity < 3:
        return "Weak: Password lacks complexity"

    # Common pattern check
    common_patterns = [
        r'123',
        r'abc',
        r'password',
        r'qwerty',
        r'111',
        r'admin',
        r'letmein',
        r'love',
        r'master',
        r'123456'
    ]

    for pattern in common_patterns:
        if re.search(pattern, password, re.IGNORECASE):
            return "Weak: Password contains common pattern"

    return "Strong: Password meets criteria for strength"

# Test the function
password = input("Enter your password: ")
print(password_strength(password))
```

Output:

Run 1:

Enter your password: Dsce@123

Weak: Password contains common pattern

Run 2:

Enter your password: Dsce11#

Weak: Password is too short

Run 3:

Enter your password: Dsce$1&@

Strong: Password meets criteria for strength

EXPT.No. 7	**File Encryption and Decryption:** Create a Python program that encrypts and decrypts files using cryptographic algorithms such as AES (Advanced Encryption Standard).	DATE:

Install the `pycryptodome` library

Make sure you have the `pycryptodome` library installed. If not, you can install it using pip:

pip install pycryptodome

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

def encrypt_content(plain_text: bytes, key: bytes) -> bytes:
    iv = get_random_bytes(16)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded_data = pad(plain_text, AES.block_size)
    encrypted_data = cipher.encrypt(padded_data)
    return iv + encrypted_data

def decrypt_content(encrypted_data: bytes, key: bytes) -> bytes:
    iv = encrypted_data[:16]
    encrypted_data = encrypted_data[16:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded_data = cipher.decrypt(encrypted_data)
    plain_text = unpad(padded_data, AES.block_size)
    return plain_text

def read_file(file_path: str) -> bytes:
    with open(file_path, 'rb') as f:
        return f.read()

def write_file(file_path: str, data: bytes):
    with open(file_path, 'wb') as f:
        f.write(data)

def main():
    key = b'This is a key123'  # 16 bytes key for AES-128

    # Read the original file content
```

```
    original_content = read_file('example.txt')
    print("Original content:")
    print(original_content.decode())

    # Encrypt the content
    encrypted_content = encrypt_content(original_content, key)
    print("\nEncrypted content (hex):")
    print(encrypted_content.hex())

    # Decrypt the content
    decrypted_content = decrypt_content(encrypted_content, key)
    print("\nDecrypted content:")
    print(decrypted_content.decode())

    # Write the encrypted and decrypted files
    write_file('encrypted_example.bin', encrypted_content)
    write_file('decrypted_example.txt', decrypted_content)

if __name__ == "__main__":
    main()
```

Explanation

1. **Functions**:
    - `encrypt_content`: Encrypts the given plain text using AES in CBC mode with the given key and returns the IV concatenated with the encrypted data.
    - `decrypt_content`: Decrypts the given encrypted data using AES in CBC mode with the given key and returns the original plain text.
    - `read_file`: Reads the content of a file as bytes.
    - `write_file`: Writes bytes data to a file.
2. **Main Function**:
    - Reads the content of `example.txt`.
    - Prints the original content.
    - Encrypts the content and prints the encrypted data in hexadecimal format.
    - Decrypts the encrypted content and prints the decrypted content.
    - Writes the encrypted data to `encrypted_example.bin` and the decrypted content to `decrypted_example.txt`.

Running the Program

1. **Create the Input File**:
    - Create a file named `example.txt` with the following content:

      ```
      sql
      Copy code
      ```

```
This is a test file.
It contains some sample text.
```

2. **Run the Program**:
    - Execute the script to see the original, encrypted, and decrypted content printed in the console.

Console Output Example:

Original content:
This is a test file.
It contains some sample text.

Encrypted content (hex):
d3d14e98e8fae58b530f5a43145c1e5e913d3a4bc8d2b0c84d8b72602b6bff94e208e7173f7fef054d5785b31543b9f7

Decrypted content:
This is a test file.
It contains some sample text.

EXPT.No. 8	**Network Packet Sniffer:** Build a packet sniffing tool in Python that captures and analyses network traffic on a local network.	DATE:

```
import scapy.all as scapy

def sniff_packets(interface, filter=None):
    scapy.sniff(iface=interface, store=False, prn=process_packet, filter=filter)

def process_packet(packet):
    if packet.haslayer(scapy.IP):
        ip_src = packet[scapy.IP].src
        ip_dst = packet[scapy.IP].dst
        protocol = packet[scapy.IP].proto
        if packet.haslayer(scapy.TCP):
            src_port = packet[scapy.TCP].sport
            dst_port = packet[scapy.TCP].dport
            print(f"TCP Packet: {ip_src}:{src_port} --> {ip_dst}:{dst_port}")
        elif packet.haslayer(scapy.UDP):
            src_port = packet[scapy.UDP].sport
            dst_port = packet[scapy.UDP].dport
            print(f"UDP Packet: {ip_src}:{src_port} --> {ip_dst}:{dst_port}")
        else:
            print(f"IP Packet: {ip_src} --> {ip_dst}, Protocol: {protocol}")

def main():
    interface = input("Enter the interface to sniff on (e.g., 'eth0', 'wlan0'): ")
    filter = input("Enter BPF filter (optional, press Enter to skip): ")
    print("\nStarting packet sniffing...")
    sniff_packets(interface, filter)

if __name__ == "__main__":
    main()
```

Output:

TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.12:8080 --> 182.16.0.1:53962
TCP Packet: 182.16.0.12:8080 --> 182.16.0.1:53962
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.12:8080 --> 182.16.0.1:53962
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.12:8080 --> 182.16.0.1:53962
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.1:53962 --> 182.16.0.12:8080
TCP Packet: 182.16.0.12:8080 --> 182.16.0.1:53962

EXPT.No. 9	**Brute Force Attack Detector:** Develop a Python script that detects and logs brute force login attempts on a server or application.	**DATE:**

```
import re
from collections import defaultdict
from datetime import datetime

DETECTION_THRESHOLD = 5
TIME_WINDOW = 300  # 5 minutes in seconds

def parse_log_line(line):
    pattern = re.compile(r'(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}).*Login failed for
user.*from IP (?P<ip>\d+\.\d+\.\d+\.\d+)')
    match = pattern.match(line)
    return match.group('timestamp'), match.group('ip') if match else (None, None)

def convert_to_timestamp(timestamp_str):
    return datetime.strptime(timestamp_str, "%Y-%m-%d %H:%M:%S").timestamp()

def detect_brute_force(log_lines):
    failed_attempts = defaultdict(list)
    for line in log_lines:
        timestamp_str, ip = parse_log_line(line)
        if ip:
            timestamp = convert_to_timestamp(timestamp_str)
            failed_attempts[ip].append(timestamp)
            window_start = timestamp - TIME_WINDOW
            recent_attempts = [ts for ts in failed_attempts[ip] if ts >= window_start]
            if len(recent_attempts) >= DETECTION_THRESHOLD:
                print(f"Brute force detected from IP {ip} at {timestamp_str}")

# Example log lines (replace with actual log lines)
log_lines = [
    "2024-06-02 12:00:00 Login failed for user admin from IP 192.168.1.1",
    "2024-06-02 12:00:30 Login failed for user admin from IP 192.168.1.1",
    "2024-06-02 12:01:00 Login failed for user admin from IP 192.168.1.1",
    "2024-06-02 12:01:30 Login failed for user admin from IP 192.168.1.1",
    "2024-06-02 12:02:00 Login failed for user admin from IP 192.168.1.1",
]
```

```
# Detect brute force attempts
detect_brute_force(log_lines)
```

Explanation:

1. **Log Line Parsing**: The `parse_log_line` function extracts the timestamp and IP address from each log line using regular expressions.
2. **Brute Force Detection**: The `detect_brute_force` function iterates over the log lines, extracts the timestamp and IP address for each failed login attempt, and maintains a dictionary of failed attempts per IP address. If the number of failed attempts for a specific IP address exceeds the detection threshold, it prints a message indicating a brute force attempt.
3. **Example Log Lines**: The `log_lines` variable contains sample log lines. Replace these with actual log lines from your application or server.
4. **Detect Brute Force Attempts**: The `detect_brute_force` function is called with the example log lines to demonstrate the detection process.
5. **Timestamp Parsing**: The regular expression correctly extract the timestamp in the format `YYYY-MM-DD HH:MM:SS`.
6. **Timestamp Conversion**: A function `convert_to_timestamp` used to convert the timestamp string to a Unix timestamp (seconds since the epoch).
7. **Threshold Check**: Instead of comparing timestamps directly, we compare the number of recent attempts within the time window to the detection threshold.

Output:

```
Brute force detected from IP 192.168.1.1 at 2024-06-02 12:02:00
```

This output indicates that a brute force attempt was detected from the IP address `192.168.1.1` at the timestamp `2024-06-02 12:02:00`, which matches the last log line in the sample data.

EXPT.No. 10	**Security Incident Reporting Tool:** Develop a Python-based incident reporting tool where employees can report security incidents such as lost devices, suspicious emails, or physical security breaches.	**DATE:**

```
import sqlite3
from datetime import datetime

# Initialize SQLite database
def initialize_database():
    conn = sqlite3.connect('incident_reports.db')
    cursor = conn.cursor()
    cursor.execute('''CREATE TABLE IF NOT EXISTS incidents
                      (id INTEGER PRIMARY KEY,
                      timestamp TEXT,
                      category TEXT,
                      description TEXT,
                      reporter TEXT)''')
    conn.commit()
    conn.close()

# Submit incident report
def submit_incident(category, description, reporter):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    conn = sqlite3.connect('incident_reports.db')
    cursor = conn.cursor()
    cursor.execute('''INSERT INTO incidents (timestamp, category, description, reporter)
                      VALUES (?, ?, ?, ?)''', (timestamp, category, description, reporter))
    conn.commit()
    conn.close()

# View all incident reports
def view_incidents():
    conn = sqlite3.connect('incident_reports.db')
    cursor = conn.cursor()
    cursor.execute('''SELECT * FROM incidents''')
    incidents = cursor.fetchall()
    conn.close()
    return incidents
```

```
# User interface
def main():
    initialize_database()

    while True:
        print("\nIncident Reporting Tool")
        print("1. Report an Incident")
        print("2. View All Incidents")
        print("3. Exit")
        choice = input("Enter your choice: ")

        if choice == '1':
            category = input("Enter the category of incident: ")
            description = input("Enter a brief description: ")
            reporter = input("Your name: ")
            submit_incident(category, description, reporter)
            print("Incident reported successfully!")

        elif choice == '2':
            incidents = view_incidents()
            print("\nAll Incidents:")
            for incident in incidents:
                print(f"ID: {incident[0]}, Timestamp: {incident[1]}, Category: {incident[2]},
Description: {incident[3]}, Reporter: {incident[4]}")

        elif choice == '3':
            print("Exiting...")
            break

        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Explanation:

1. The script provides a simple command-line interface for employees to report incidents and administrators to view all reported incidents.
2. Incident reports are stored in a SQLite database with fields for timestamp, category, description, and reporter's name.
3. The `initialize_database` function creates the SQLite database and incident reports table if they don't exist.
4. The `submit_incident` function adds a new incident report to the database with the current timestamp.
5. The `view_incidents` function retrieves all incident reports from the database.
6. The `main` function is the entry point of the program and displays a menu for users to report incidents, view all incidents, or exit the tool.

Output:

```
Incident Reporting Tool
1. Report an Incident
2. View All Incidents
3. Exit
Enter your choice: 1
Enter the category of incident: Last Devices
Enter a brief description: Last company laptop in the cafeteria
Your name: John Doe
Incident reported successfully!

Incident Reporting Tool
1. Report an Incident
2. View All Incidents
3. Exit
Enter your choice: 2

All Incidents:
ID: 1, Timestamp: 2024-06-02 12:13:55, Category: Last Devices,
Description: Last company laptop in the cafeteria, Reporter: John Doe
Incident Reporting Tool
1. Report an Incident
2. View All Incidents
3. Exit
Enter your choice: 3
Exiting...
```